

TNT: 改进分块训练以实现测试时记忆

Zeman Li^{1,2*} Ali Behrouz² Yuan Deng² Peilin Zhong² Praneeth Kacham² Mahdi Karami²
Meisam Razaviyayn^{1,2} Vahab Mirrokni²

¹University of Southern California ²Google Research

摘要

具有深度测试时记忆模块的循环神经网络（RNN），如泰坦（Titans）和测试时训练（TTT），代表了一种有前景的、线性扩展的范式，与变换器（Transformer）不同。尽管这些表达性模型尚未达到最先进变换器的峰值性能，但由于训练速度极慢和硬件利用率低，其潜在在很大程度上尚未被开发。现有的并行化方法迫使一个由块大小超参数支配的根本性冲突：大块提升速度但降低性能，因此需要固定的、次优的折中方案。为了解决这一挑战，我们提出了 TNT，这是一种新颖的训练范式，通过两阶段过程将训练效率与推理性能解耦。第一阶段是注重效率的预训练阶段，利用分层记忆。一个全局模块处理大型、硬件友好的块以获取长程上下文，而多个并行的局部模块处理细粒度细节。关键的是，通过周期性重置局部记忆状态，我们打破了顺序依赖，从而实现了大规模上下文并行化。第二阶段是短暂的微调阶段，仅将局部记忆模块调整到更小、更高分辨率的块大小，以最小的开销最大化准确率。在泰坦和测试时训练模型上评估，TNT 实现了训练速度的大幅提升——比最准确的基线配置快达 $17 \times$ 倍——同时提高了模型准确率。这一改进消除了一个关键的可扩展性障碍，为开发表达性循环神经网络奠定了实用基础，并促进了未来缩小与变换器性能差距的工作。

1 引言

对长序列建模的需求凸显了标准 Softmax 注意力（Vaswani et al. 2017）的一个根本性局限：其二次复杂度阻碍了扩展。这激发了大量研究，以寻求更高效的架构。

在这些新兴范式中，一种尤为强大的方法植根于测试时记忆（Test-time Memorization）（Sun, Li, et al. 2024）。利用这一原理的架构，我们称之为深度记忆模块，采用一个深度、在线适应的子网络，其参数被快速更新以编码上下文信息。突出的例子包括泰坦（Titans）（Behrouz, Zhong, et al. 2025）和阿特拉斯（Atlas）（Behrouz, Li, et al. 2025）。这种方法与线性记忆模块（Yang, Wang, Shen, et al. 2024; Yang, Wang, Zhang, et al. 2024b; Dao and Gu 2024; Sun, Dong, et al. 2023）形成鲜明对比，后者尽管效率高，但受限于矩阵值隐藏状态和线性状态转移。通过利用表达性强的非线性目标和更新规则，深度记忆模块理论上可以克服这些局限性。虽然这些方法通常尚未达到变换器（Transformer）的最先进性能，但它们代表了一种潜在有前景的高效序列建模范式，前提是其训练瓶颈能够得到解决。

尽管具有表达优势，深度记忆模块缺乏线性对应模块的高效训练算法，导致硬件利用率低。与利用硬件高效并行化的线性记忆模块不同，深度记忆模块面临来自非线性递归（例如块间的层归一化）及其深层结构复杂性的挑战。在实践中，这些挑战将其训练限制为在小数据段上进行更频繁的在线更新，导致训练中的计算吞吐量差。这造成了一种内在的张力，因为这些

*Work 在谷歌研究院实习期间完成。†{zemanli, alibehrouz, dengyuan, pkacham, mahdika, razaviyayn, mirrokni}@google.com, 以及 peilin.zhong@columbia.edu

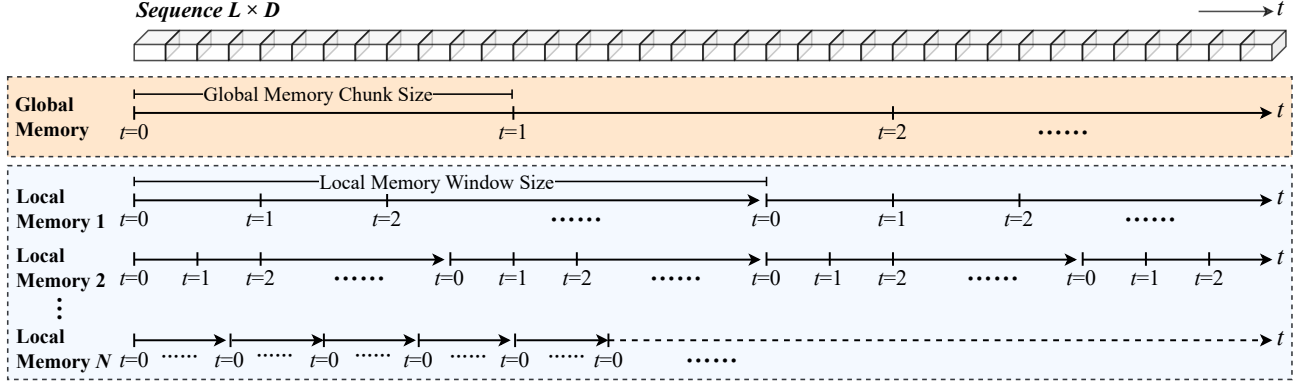


图 1: 说明 TNT 记忆层级的基本示意图。在每一行中, 相同 t 值的更新同时运行 (并行执行)。 $t = 0$ 是记忆的初始化。

模型通常依赖固定的、较小的块大小 (例如 16 到 64 个标记) 来平衡记忆层表达能力和训练效率。因此, 上下文学习能力与计算性能之间的这种权衡已成为阻止这些模型在实践中应用于真正长序列的关键瓶颈。解决这一根本张力是本文的主要目标。

近期工作试图缓解这一问题。Zhang, Bi 等人 (2025) 将大块数据与局部注意力相结合以增强并行性。然而, 这绕开了低效问题而非解决它, 通过混合记忆与注意力使分析复杂化, 并忽视了推理阶段对小块数据 (理想为 1) 的需求。与此同时, Guo 等人 (2025) 提出了一种分层记忆系统, 但该系统仅限于线性记忆模块, 且不支持短期记忆。

为解决这一矛盾, 我们提出 TNT₁, 一种用于深度记忆模块的新型训练范式。我们的核心洞见是: 模型的不同组件应在不同的训练阶段以不同的粒度处理信息。TNT 首先进行以效率为中心的预训练阶段, 旨在最大化吞吐量。这通过分层记忆系统实现: 全局记忆模块在大块、硬件友好的数据上运行以捕获长程上下文, 而多个局部记忆模块并行处理细粒度细节。关键的是, 我们为局部记忆状态引入周期性重置机制。这打破了即使是非线性循环神经网络 (例如, 在步骤间进行归一化的网络) 中固有的顺序依赖, 从而实现大规模上下文并行化。这是一项关键创新, 因为跨序列长度高效并行化非线性循环是一个长期挑战, 在变换器和专门的线性循环神经网络 (其中可应用并行扫描) 之外基本未解决。随后, 以性能为中心的微调阶段将模型调整为最优推理状态。在此阶段, 仅调整局部记忆模块以使用更小的块大小, 以最小的额外计算成本实现高分辨率准确率。这种两阶段方法有效地将训练效率与推理性能解耦, 显著提高了训练可扩展性, 同时解决了先前架构的一个关键局限性。此外, 局部记忆系统本身可以是分层的, 采用多个在不同分辨率下运行的模块。这种多分辨率方法使模型能够比单一固定块大小更有效地捕获复杂的多尺度时间动态。

TNT 是一种通用训练范式, 适用于任何深度记忆模块, 而非特定架构。通过将训练吞吐量与推理准确率解耦, 本文解决了制约先前工作的根本矛盾。这消除了对小块硬件特定优化的依赖, 并支持对架构设计空间的灵活探索。本文认为, 该范式将为取代 Softmax 注意力开辟新的研究方向。

本文的主要贡献总结如下:

- 本文识别出限制深度记忆模块可扩展性与性能的三个根本挑战: 1) 记忆压缩与检索之间的领域不匹配; 2) 记忆性能与计算效率之间的权衡; 3) 训练与推理之间的块大小不匹配 (第 3 节)。
- 本文引入 Q-K 投影 (Q-K Projection), 一种高效机制, 用于解决记忆压缩与检索之间的领域不匹配问题 (第 4.1.2 节)。
- 本文提出一种新颖的分层记忆架构, 并引入周期性状态重置, 从而支持上下文并行

¹TNT 可视为 Titans iNside Titans 或 TTT iNside TTT 的缩写, 亦暗示其对训练效率的“爆炸性”影响。

非线性深度记忆模块（第 4.1 节）。

- 本文提出一种高效的微调机制，以解决深度记忆模块中训练与推理之间块大小不匹配的问题（第 4.2 节）。
- 综合上述内容，我们提出了 TNT，一种通用的两阶段训练范式，通过结合高效预训练与高分辨率微调，将训练效率与推理性能解耦（图 1、图 3、第 4 节）。
- 我们在 Titans 架构上验证了 TNT，实现了高达 $17.37\times$ 的训练加速，同时提升了准确率，显著推进了表达性循环神经网络（RNN）的实用性（第 5 节）。

问题定义与符号表示 我们的目标是训练一个参数为 $\theta \in \mathbb{R}^{d_m}$ 的神经网络来执行下一词元预测。给定一个序列 $\mathbf{x} = (x_1, \dots, x_L)$ ，模型的目标是利用其前文词元 $(x_1, \dots, x_{t-1})$ 的上下文来预测每个词元 x_t 。遵循注意力机制的形式，每个词元 x_t 由一个 d 维向量表示。每个输入词元 x_t 被投影为查询、键和值向量： $q_t, k_t, v_t \in \mathbb{R}^d$ 。为了便于后续分块操作中的符号表示，我们定义一个函数 $\xi(i, j) := i - (i \bmod j)$ ，该函数对于块大小 j ，找到包含索引 i 的块的起始位置。

2 预备知识

本节回顾预备知识。扩展的相关工作见附录 A。

2.1 基于测试时记忆的深度记忆模块

序列建模的一种强大范式是测试时记忆（Test-Time Memorization）（Sun, Li, et al. 2024），它通过引入一个辅助的、快速适应的神经网络来增强模型。与主模型参数（即“慢权重”， θ 仅在训练期间更新）不同，该方法引入了“快权重”（Schlag et al. 2021）。这些快权重，记为 W ，参数化一个子网络， $f(W, \cdot) : \mathbb{R}^d \rightarrow \mathbb{R}^d$ ，该子网络在训练和推理过程中根据输入词元在线更新，以动态存储上下文信息。尽管这些模块与变换器（Transformer）相比尚未取得最先进的结果（Arora et al. 2024; Behrouz, Li, et al. 2025），但提高其训练效率对于实现缩小这一差距所需的更广泛实验至关重要。

本文聚焦于一个相似/相关的原理：深度记忆模块（Deep Memory Modules）（Irie 等人，2021；Sun, Li 等人，2024；Behrouz, Zhong 等人，2025；Behrouz, Li 等人，2025；Behrouz, Razaviyayn 等人，2025a）。与以线性状态转移为特征的线性记忆模块（Linear Memory Modules）（Sun, Dong 等人，2023；Yang, Wang, Zhang 等人，2024b；Dao 和 Gu，2024；Karami 和 Mirrokni，2025；Hu, Pan 等人，2025）相比，深度记忆模块采用非线性递归规则和复杂的记忆结构。

深度记忆模块的核心机制可提炼为对每个输入标记依次执行的两个操作：1. 记忆压缩（Memory Compression）和 2. 记忆检索（Memory Retrieval）。其形式化定义如下：

$$\text{Memory Compression: } W_t \leftarrow W_{t-1} - \eta_t \nabla_W \mathcal{L}(f(W_{t-1}, k_t), v_t) \quad (1)$$

$$\text{Memory Retrieval: } o_t = f(W_t, q_t) \quad (2)$$

在记忆压缩中，快速权重 W 通过梯度下降进行更新，由自监督损失 $\mathcal{L}(\cdot, \cdot)$ （例如均方误差）和学习得到的学习率 η_t 引导。该目标将变换后的键 $f(W_{t-1}, k_t)$ 与其值 v_t 关联起来，将信息压缩到固定大小的神经记忆中（Wang 等人，2025；Behrouz, Razaviyayn 等人，2025b）。在记忆检索中，更新后的 W_t 处理查询 q_t 以产生 o_t 。这两个操作对每个标记迭代执行。

2.2 分块并行训练

公式 1-2 中的顺序依赖（ W_t 依赖于 W_{t-1} ）阻碍了跨序列长度的并行化。为解决这一问题，深度记忆模块采用分块并行训练（Chunkwise Parallel Training）（Hua 等人，2022；Sun, Dong 等人，2023）以实现硬件高效的训练。

其核心原理是将输入序列划分为大小为 C 的非重叠块。在每个块内，针对该块起始时的快速权重状态，计算每个标记损失梯度的近似值。这种公式化打破了梯度计算中标记间的顺序依赖，使得块内所有标记的更新可以并行计算。对于时间步 t 处的标记，其形式化操作如下：

$$\text{Chunkwise Memory Compression: } W_t \leftarrow W_{\xi(t,C)} - \sum_{\tau=\xi(t,C)}^t \eta_{\tau} \nabla_W \mathcal{L}(f(W_{\xi(t,C)}, k_{\tau}), v_{\tau}) \quad (3)$$

$$\text{Chunkwise Memory Retrieval: } o_t = f(W_t, q_t) \quad (4)$$

此处， $W_{\xi(t,C)}$ 表示包含词元 t 的块开始时快速权重的状态（参见第 1 节末尾对 $\xi(\cdot, \cdot)$ 的定义）。尽管获取 W_t 的更新仍依赖于其块内的先前词元，但梯度的求和可以利用并行操作（例如累积求和）高效实现，从而显著提升训练期间的硬件利用率。然而，仍存在顺序依赖：第 n 个块的快速权重最终状态 W_{nC} 被用作第 $(n+1)$ 个块的初始状态。

3 深度记忆模块面临的挑战

尽管分块并行化使深度记忆模块能够在长序列上进行训练，但这一范式引入了显著挑战，限制了其实际性能和可扩展性。本节将概述深度记忆模块的三个根本性挑战。

挑战一：缺乏高效的训练实现。深度记忆模块面临的首要挑战是其训练过程效率低下，导致硬件利用率不佳。虽然分块并行化在理论上能够实现二次方扩展，但在实践中，训练吞吐量显著落后于线性记忆模块。这种差距源于模型表达能力与计算效率之间的根本性矛盾。

为了保持细粒度的学习信号，深度记忆模块需要较小的块大小（例如 16-64 个词元）（Sun, Li, et al. 2024），这无法充分利用加速器，使得训练受限于内存带宽（而非计算能力）。虽然线性记忆模块使用定制核函数（例如利用静态随机存取存储器）（Sun, Dong, et al. 2023; Gu et al. 2023; Qin et al. 2024; Yang, Wang, Shen, et al. 2024; Yang, Wang, Zhang, et al. 2024a），但这依赖于线性状态转移，与深度记忆模块的大型非线性状态不兼容。

其后果是深度记忆模块的浮点运算利用率极低，通常低于硬件峰值性能的 5-10%（Zhang, Bi, et al. 2025）。这种严重的低效使得预训练极其缓慢且成本高昂，构成了一个主要瓶颈，削弱了这些表达力强的模型在真正长序列上的实际应用。

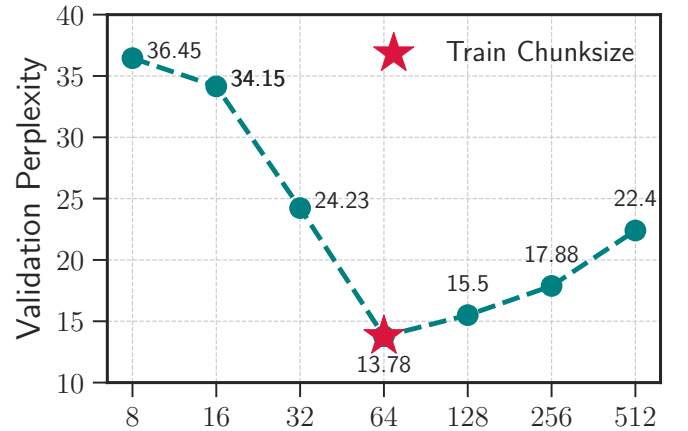


图 2：在预训练时使用 $C = 64$ 的 550M Titans 模型上，推理块大小的敏感性。当推理块大小与训练块大小一致时，性能达到最优。

挑战二：记忆压缩与检索之间的不一致性。记忆子网络的训练方式与使用方式之间存在根本性的不一致。在记忆压缩（公式 1）过程中，子网络 $f(W, \cdot)$ 通过将键（ k_t ）与值（ v_t ）关联起来，被优化以学习从键空间到值空间的映射。然而，在记忆检索（公式 2）过程中，网络使用查询向量（ q_t ）而非键进行查询。这种替换违背了所学函数的预期输入域，造成训练目标与检索任务之间的差异。这种域偏移可能降低所学映射的完整性，并限制模型的检索性能。我们的实证验证见第 5.4 节。

挑战三：性能对固定预训练块大小的敏感性。块大小超参数 C 控制着训练吞吐量与模型表达能力之间的权衡。当前深度记忆模块的实践是在预训练和推理阶段使用相同的固定块大小。然而，我们观察到推理时的性能对该预训练选择高度敏感。例如，如图 2 所示，以块大小 64 预训练的模型仅在使用相同块大小评估时才能达到最佳困惑度。

这一结果揭示了关键的训练-测试不匹配，并违背了直觉，即推理时使用更小的块应通过以“更新鲜”的学习信号捕获细粒度依赖关系而产生更优性能。相反，模型变得过度专门化于训练期间所见的具体块分辨率。这种不灵活性是一个显著的局限性；理想情况下，以较大、硬件友好的块大小预训练的模型应具有足够的适应性，以便在推理时使用更小、更精确的块大小表现得更好。当前的深度记忆模块无法实现这种适应性。

4 TNT：一种改进的深度记忆训练框架

为解决第 3 节中概述的挑战，我们提出 TNT，一种改进的深度记忆模块训练范式。我们的框架围绕一个两阶段过程构建，旨在解决训练效率与推理性能之间的内在矛盾：一个以效率为重点的预训练阶段和一个以性能为重点的微调阶段。

第一阶段通过引入一种新颖的分层记忆架构来最大化训练吞吐量，该架构实现了前所未有的并行性，直接解决了硬件利用率低和记忆目标不一致的挑战（挑战 1-2）。第二阶段采用高效的微调策略，使模型适应高分辨率、小块推理，解决了对预训练块大小的敏感性（挑战 3）。这种两阶段方法有效地将训练效率与推理性能解耦，克服了先前深度记忆架构的一个关键局限性。

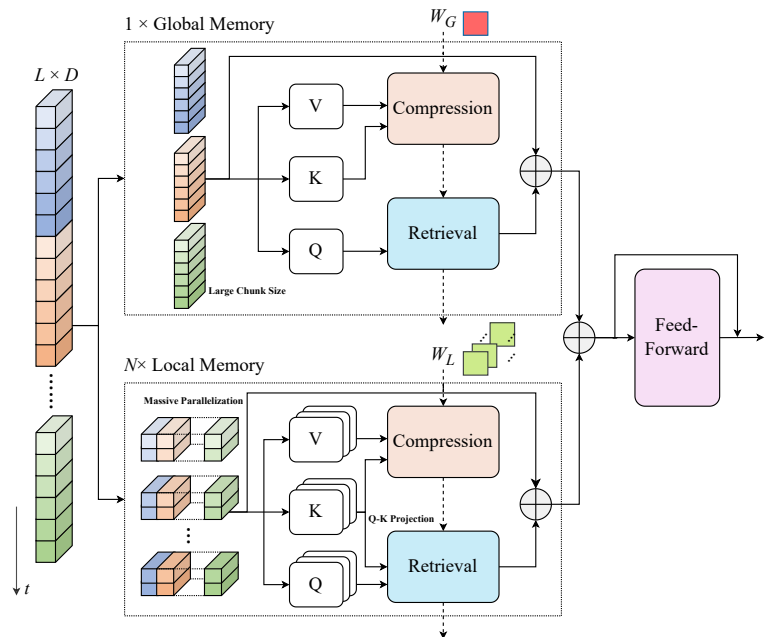


图 3：TNT 第一阶段架构概览。

4.1 TNT 第一阶段：以效率为中心的预训练

4.1.1 TNT 记忆压缩：分层记忆

顺序状态依赖性阻碍了上下文并行（跨设备并行处理序列分片）。为了实现这一点，本文提出所有并行分片使用相同的学习状态初始化其局部记忆， W_{init} 。这打破了块间依赖性，允许大规模并行化。然而，这导致局部记忆模块丢失全局上下文。为了解决这个问题，本文引入了一个全局记忆模块，由 v 参数化，与分片的局部记忆并行运行。全局记忆以相对较大的块大小（例如，2048 或更大）处理序列，使其能够高效地捕获长距离依赖性，同时保持高硬件利用率。这创建了一个分层系统，其中局部记忆处理并行分片内的细粒度信息，而全局记忆提供总体上下文。

这种分层结构是灵活的：一个模型可以设计为具有 1 个全局记忆模块和 N 个局部记忆模块，每个模块以不同的分辨率运行。为了清晰说明，本文将假设最简单的情况，其中 $N = 1$ 。本文将 TNT 的广义公式推迟到附录 E。现在正式定义记忆压缩机制。

TNT 记忆压缩规则。分层记忆按如下方式更新：

全局记忆更新。全局记忆状态 V 以较大的块大小 C_G 沿输入顺序演化。

$$V_{(n+1)C_G} \leftarrow V_{nC_G} - \sum_{t=nC_G}^{(n+1)C_G} \eta_t \nabla_V \mathcal{L}(f(V_{nC_G}, k_t), v_t) \quad n = \{0, \dots, L//C_G\} \quad (5)$$

局部记忆更新。局部记忆 W 在长度为 S_L 的序列分片上并行操作。在每个分片内，更新使用较小的块大小 C_L 。

$$W_t \leftarrow \begin{cases} W_{\text{init}} & \text{if } 0 \equiv t \pmod{S_L} \\ W_{t-1} - \sum_{\tau=\xi(t, C_L)}^t \eta_\tau \nabla_W \mathcal{L}(f(W_{\xi(t, C_L)}, k_\tau), v_\tau) & \text{Otherwise} \end{cases} \quad (6)$$

全局记忆更新（公式 5）遵循标准的逐块公式，其中状态在大块之间顺序传递。为了最大化训练吞吐量，全局块内所有标记的梯度均相对于该块的初始状态计算，从而实现高度并行的更新。

相比之下，局部记忆更新（公式 6）引入了本文的关键创新：周期性状态重置。该规则强制局部记忆状态 w_t 在每个长度为 S_L 的片段开始时重置为共享的可学习初始状态 W_{init} 。这种周期性重置是打破输入中长程顺序依赖的关键机制，从而为细粒度局部记忆模块实现真正的上下文并行。

深度记忆模块的层次化设计通过双管齐下的方法提升训练效率。全局模块通过处理大块来创建硬件饱和、计算密集型的操作。同时，局部记忆的重置机制支持上下文并行，其中序列被处理为独立的块，可以分布在多个设备上或堆叠在单个加速器上，以大幅提高训练吞吐量。

4.1.2 TNT 记忆检索：Q-K 投影

如挑战 2 所述，记忆压缩步骤（公式 6）优化 $f(W, \cdot)$ 以将键空间映射到值空间。然而，在检索时，网络使用查询向量 q_t 进行查询，该向量可能位于已学习的键域之外，从而降低性能。

为解决这一问题，本文提出 Q-K 投影方法：将查询 q_t 投影到由先前观测到的键所张成的子空间上。这确保了输入到记忆函数的数据处于记忆训练时所处的空间中。最终输出结合了来自全局记忆（原始查询）和局部记忆（投影查询）的检索结果。本文仅在局部应用投影，因为其细粒度特性使其对不匹配更为敏感。

TNT 记忆检索规则。分层记忆的检索方式如下：

$$o_t = f(V_{\xi(t, C_G)}, q_t) + f\left(W_t, \sum_{\tau=\xi(t, C_L)}^t \frac{k_\tau k_\tau^\top}{\|k_\tau\|^2} q_t\right) \quad (7)$$

关键在于，这种 Q-K 投影无需存储所有历史键，否则计算和内存开销将难以承受。相反，投影矩阵 $\sum_{\tau=1}^t \frac{k_\tau k_\tau^\top}{\|k_\tau\|^2} \bar{k}_\tau \in \mathbb{R}^{d \times d}$ 可以以累加和的形式维护。这形成了一个恒定大小的状态，能够以分块并行的方式高效更新。由于许多现代深度记忆模块会通过 L2 范数对查询 (q_t) 和键 (k_t) 向量进行归一化，Q-K 投影中的分母可简化为 $\sum_{\tau=1}^t k_\tau k_\tau^\top$ 。关于这一高效实现的更多细节见附录 C。

4.2 TNT 第二阶段：面向性能的细粒度微调

在第一阶段解决训练效率问题后，本文转向优化推理性能。一种直观的提升模型分辨率的方法是使用更小的块大小来评估预训练模型。然而，如挑战 3 所述，预训练与评估块大小之间的直接不匹配会导致显著的性能下降。

本文的核心洞见在于，这种训练与测试之间的差异可以以极小的计算开销加以纠正。实验观察到，一个简短的微调阶段——使用更小的局部记忆块大小对预训练模型进行少量步骤的更新——不仅能恢复原有性能，而且常常超越原有水平。

基于这一发现，本文引入 TNT 框架的第二阶段：面向性能的微调阶段。在此阶段，本文使用更小的局部块大小（ C' ）继续训练高效预训练得到的模型。这一过程使模型适应推理所需的 $< C_L$ 高分辨率，而成本仅为预训练的一小部分。通过这种方式，第二阶段直接解决了挑战 3，弥合了高效训练所需的大块大小与推理时最优性能所需的小块大小之间的差距。

这一两阶段过程将预训练效率与推理需求解耦。大部分训练采用最大吞吐量（大块），而最终模型以最小开销生成。此外，微调使模型专门针对理想推理场景：局部块大小为 1（ $C'_l = 1$ ）。这与自回归生成的标准预填充-解码范式一致。全局记忆处理上下文预填充，优化后的局部记忆处理迭代解码。

5 实验

我们实证评估了两阶段训练框架 TNT。虽然 TNT 与模型无关，但我们将其实例化于强大的深度记忆模型 Titans（Behrouz, Zhong, et al. 2025）上，以证明其有效性。我们在实验中验证了关于训练时间和模型准确率的声明。

5.1 实验设置

基线。我们与几种强大的长上下文架构进行比较。主要比较对象是 Titans（Behrouz, Zhong, et al. 2025），即我们的基础模型。我们还与普通 Transformer（Vaswani et al. 2017）、门控 Transformer（Qiu et al. 2025）和 TTT（Sun, Li, et al. 2024）进行基准测试。

训练与 TNT 配置。我们按照（Behrouz, Zhong, et al. 2025）训练 1.5 亿参数模型，使用 T5 分词器（32k 词汇表）。我们使用 AdamW 优化器（Loshchilov et al. 2017），权重衰减为 0.1，采用余弦调度（峰值学习率 1×10^{-3} ）。实验在 TPUV5 pod 上进行（2x2x2 拓扑，模型并行度 2）。对于 TNT， N 局部模块配置由其块大小 $\tilde{C}_L = \{C_{L,1}, \dots, \tilde{C}_{L,N}\}$ 表示。例如， $C_L = \{8, 16\}$ 表示两个局部模块，块大小分别为 8 和 16。全局记忆使用 $C_G = 2048$ 。

Experimental Configurations. For efficiency benchmarks (Sec. 5.2), we vary context length (2k-32k) with a 0.5M token batch size and local window $S_L = 2048$. For performance evaluation (Sec 5.3), we use a 16k context length, 1M token batch size, and $S_L = 4096$.

5.2 利用 TNT 实现更快的记忆训练

运行时间随序列长度线性扩展。本文首先通过固定每批次的标记数量、改变序列长度来分析单步运行时间性能。如图 4 所示，TNT 的运行时间随序列长度线性增长，与 Titans 和标准注意力的二次增长形成对比。这种扩展优势在长上下文场景下尤为显著。在 32K 序列长度下，TNT 比具有相同记忆块大小（ $C_L = C = 16$ ）的可比 Titans 模型快 **5.1x**。本文还观察到，更大的局部块大小持续提升 TNT 的速度；使用 $C_L = \{128\}$ 时，TNT 比高度优化的 FlashAttention 核函数（Dao 2024）快 **1.3x**。

TNT 高度可并行化的架构实现了随序列长度线性扩展的运行时间，这是相对于标准注意力二次复杂度的一个关键优势。尽管 Titans 等模型在理论上也是线性的，但其固有的顺序依赖性阻碍了有效的并行化，导致硬件利用率低下和实际运行时间更长。

表 1: TNT 达到目标训练损失的速度比基线 Titans 快至 17 $\times$ 。该表比较了不同 150M 模型达到相同目标损失 3.20 所需的时间。

Models	Implementation	C or C_L	Training Time (hrs)	Speedup
Titans	JAX	8	19.48	1.00 $\times$
Titans	JAX	16	10.79	1.81 $\times$
Titans	JAX	32	6.45	3.02 $\times$
Titans	JAX	64	4.18	4.67 $\times$
Titans	JAX	128	3.71	5.25 $\times$
Transformer (w/o gating)	JAX	-	1.74	11.18 $\times$
Transformer (w gating)	JAX	-	1.38	14.10 $\times$
Transformer (w/o gating)	FlashAttention (Pallas)	-	1.23	15.90 $\times$
Transformer (w gating)	FlashAttention (Pallas)	-	0.96	20.22 $\times$
TNT	JAX	{8}	2.54	7.68 $\times$
TNT	JAX	{16}	1.65	11.78 $\times$
TNT	JAX	{32}	1.22	15.92 $\times$
TNT	JAX	{64}	1.12	17.37 $\times$
TNT	JAX	{128}	1.16	16.75 $\times$

在长序列上。随着序列长度的增加，TNT 优越的可扩展性创造了一个交叉点，使其变得显著更快。这种效率在非常长的上下文场景下最为突出；例如，在序列长度为 32 K 时，TNT 的原生 JAX 实现（ $C_L = 128$ ）甚至优于高度优化的 FlashAttention 核函数，证实了其适用于要求苛刻的长上下文训练场景。

时间-质量比较。我们接下来将这些单步运行时的提升转化为实用的时间-质量设定。如表 1 所示，我们的 TNT 框架显著加速了达到目标模型质量所需的总训练时间。我们最佳配置实现这一目标的速度比原始 Titans 基线快

17.4 $\times$ 。这种效率提升是架构的根本特性；例如，使用相同的局部记忆块大小 8，TNT 已经比其 Titans 对应模型快 7.7 $\times$ 。虽然在 JAX 中与标准普通变换器相比具有竞争力，但我们的实现尚未超越高度优化的基线，如带 FlashAttention 的门控变换器（Dao, Fu, et al. 2022）。这是一个预期结果，因为 TNT 目前缺乏自定义核函数，我们将其留作未来工作。尽管如此，这些结果确立了 TNT 作为循环模型研究的高效基础，并有一条清晰的路径来匹配最先进变换器的速度。**5.3 TNT 提升模型质量** 我们的 TNT 框架显著提升了模型质量，超越了强大的基于 RNN 的基线和标准变换器实现。如表 2 所述，初始的第 1 阶段预训练本身就非常有效。我们最佳的第 1 阶段模型达到了 **23.13** 的平均困惑度，相比表现最好的 Titans 模型（25.07）和普通变换器（23.58）有显著改进。虽然 TNT 的困惑度尚未完全匹配最先进的门控变换器（22.39），但它在常识推理任务上取得了更高的平均准确率（**41.0%** 对 39.7%）。在此规模下，我们认为困惑度是语言建模能力更稳定的度量，因为下游任务准确率可能受到更高方差的影响。

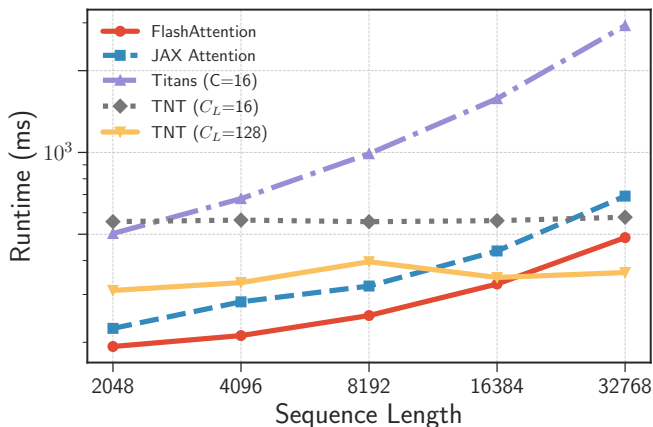


图 4: 不同模型和实现在不同序列长度下的运行时间比较，每批次标记数量固定为 0.5M。更多结果见图 5。

此外，第 2 阶段微调过程提供了一种高效的方法进一步提升性能。该阶段计算成本低廉，仅需原始预训练计算量的额外 5%（见表 4），然而它

表 2: TNT (1.5 亿参数) 和基线在语言建模和常识推理任务上的性能, 分别在 100 亿词元上训练。对于 TNT 模型, 全局块大小固定为 $C_G = 2048$, C_L , 表示局部记忆模块使用的块大小集合。块内最佳结果已高亮显示。详细训练时间见表 4。

Model	C or C_L	C4 ppl ↓	FineWeb ppl ↓	PG19 ppl ↓	Avg. ppl ↓	PIQA acc ↑	Hella. acc ↑	ARC-e acc ↑	CSQA acc ↑	Avg. acc ↑
150M params / 10B tokens										
Transformer (w/o gating)	-	20.98	20.59	29.18	23.58	62.0	30.9	34.8	25.5	38.3
Transformer (w gating)	-	19.82	19.61	27.75	22.39	63.3	32.2	36.8	26.7	39.7
TTT (2024)	256	24.18	24.31	34.36	27.62	60.6	30.8	34.1	26.9	38.1
Titans (2025) ²	256	23.53	24.13	33.73	27.13	61.3	30.8	35.1	27.8	38.8
Titans	8	22.25	22.07	30.90	25.07	60.8	32.0	35.5	27.8	39.0
TNT Stage 1: Efficiency-Focused Pre-training										
TNT Stage 1	{8}	21.04	21.01	30.24	24.10	61.8	32.8	37.4	30.3	40.6
	{8,16}	20.74	20.73	29.94	23.80	63.5	32.4	37.4	30.6	41.0
	{4,8,16}	20.47	20.43	29.43	23.44	62.9	32.4	36.4	28.9	40.2
	{4,8,16,32}	20.15	20.17	29.08	23.13	63.2	32.0	36.7	30.3	40.6
TNT Stage 2: Performance-Focused Fine-tuning on Stage 1 models										
TNT Stage 2	{1}	20.86	20.91	30.21	23.99	63.2	32.8	37.4	30.1	40.9
	{2,4}	20.65	20.70	29.97	23.77	63.4	32.5	37.3	30.2	40.9
	{2,4,8}	20.32	20.35	29.42	23.36	64.0	32.0	36.9	28.1	40.3
	{2,4,8,16}	20.10	20.13	29.05	23.09	63.5	32.3	37.4	30.2	40.9

持续将平均困惑度降低至最终值 **23.09**。这些结果验证了 TNT 作为一种有效框架, 能够生成超越先前基于循环神经网络架构局限性的高质量模型, 并成为标准变换器 (Transformer) 的有力替代方案。

5.4 消融实验

我们进行了消融实验以验证 TNT 的关键设计选择, 结果汇总于表 3。

分层记忆。分层设计的有效性显而易见。逐步增加局部记忆模块持续提升性能, 相较于 Titans 基线, 困惑度从 23.53 降至 20.15 (使用四个局部模块)。相反, 移除全局记忆则有害 (困惑度升至 25.60), 证实了其在捕获长程依赖中的关键作用, 这些依赖因局部记忆的重置机制而丢失。

查询-键投影。查询-键投影对性能至关重要。移除它会导致显著惩罚 (困惑度从 21.04 升至 22.01), 验证了我们的假设: 有必要缓解压缩-检索不匹配问题 (挑战 2)。

表 3: TNT 的消融实验。结果展示了每项针对深度记忆模块所提改进的贡献。

TNT	N	Language Modeling ppl ↓	C.S. Reasoning acc ↑
Base Model (Titans)	-	23.53	38.8
TNT Stage 1 (1 Global Memory)			
+1 Local Memory	1	21.04	40.6
+2 Local Memory	2	20.74	41.0
+3 Local Memory	3	20.47	40.2
+4 Local Memory	4	20.15	40.6
TNT Stage 1	1	21.04	40.6
w/o global memory	-	25.60	35.5
w/o Q-K projection	1	22.01	36.4
w Stage 2	1	20.86	40.9

第二阶段微调。应用第二阶段微调进一步增强了模型能力, 同时提升了语言建模 (20.86 困惑度) 和常识推理 (40.9%)。这表明其在使预训练模型适应高分辨率推理方面的有效性。

² 本工作基于我们对 TTT/Titans 模型的重新实现。模型在相同设置下训练, 以进行公平比较。

6 结论

本文提出 TNT，一种两阶段训练框架，解决了深度记忆模块中训练效率与推理性能之间的根本冲突。通过利用具有周期性状态重置的分层记忆架构，TNT 在预训练期间实现了大规模上下文并行，随后进行高效微调以实现高分辨率推理。实验表明，与最准确的循环神经网络基线相比，TNT 实现了高达 $17\times$ 的加速，同时提升了性能。TNT 消除了关键的可扩展性瓶颈，显著提高了深度记忆模块的实用性，并促进了未来研究以缩小与变换器的性能差距。

参考文献

- [1] Simran Arora, Sabri Eyuboglu, Michael Zhang, Aman Timalsina, Silas Alberti, James Zou, Atri Rudra, and Christopher Re. “Simple linear attention language models balance the recall-throughput tradeoff”. In: *Forty-first International Conference on Machine Learning*. 2024. URL: <https://openreview.net/forum?id=e93ffDcpH3>.
- [2] Ali Behrouz, Zeman Li, Praneeth Kacham, Majid Daliri, Yuan Deng, Peilin Zhong, Meisam Razaviyayn, and Vahab Mirrokni. “Atlas: Learning to optimally memorize the context at test time”. In: *arXiv preprint arXiv:2505.23735* (2025).
- [3] Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni. “It’s All Connected: A Journey Through Test-Time Memorization, Attentional Bias, Retention, and Online Optimization”. In: *arXiv preprint arXiv:2504.13173* (2025).
- [4] Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni. *It’s All Connected: A Journey Through Test-Time Memorization, Attentional Bias, Retention, and Online Optimization*. 2025. arXiv: 2504.13173 [cs.LG]. URL: <https://arxiv.org/abs/2504.13173>.
- [5] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. “Titans: Learning to memorize at test time”. In: *Advances in Neural Information Processing Systems* 38 (2025).
- [6] Róbert Csordás, Christopher Potts, Christopher D Manning, and Atticus Geiger. “Recurrent Neural Networks Learn to Store and Generate Sequences using Non-Linear Representations”. In: *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*. 2024, pp. 248–262.
- [7] Tri Dao. “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning”. In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=mZn2Xyh9Ec>.
- [8] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. “Flashattention: Fast and memory-efficient exact attention with io-awareness”. In: *Advances in neural information processing systems* 35 (2022), pp. 16344–16359.
- [9] Tri Dao and Albert Gu. “Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality”. In: *arXiv preprint arXiv:2405.21060* (2024).
- [10] Xavier Gonzalez, Andrew Warrington, Jimmy Smith, and Scott Linderman. “Towards scalable and stable parallelization of nonlinear rnns”. In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 5817–5849.
- [11] Albert Gu and Tri Dao. “Mamba: Linear-time sequence modeling with selective state spaces”. In: *arXiv preprint arXiv:2312.00752* (2023).
- [12] Han Guo, Songlin Yang, Tarushii Goel, Eric P Xing, Tri Dao, and Yoon Kim. “Log-linear attention”. In: *arXiv preprint arXiv:2506.04761* (2025).
- [13] Ramin Hasani, Mathias Lechner, Tsun-Hsuan Wang, Makram Chahine, Alexander Amini, and Daniela Rus. “Liquid Structural State-Space Models”. In: *The Eleventh International Conference on Learning Representations*. 2023. URL: <https://openreview.net/forum?id=g40TKRKfS7R>.
- [14] Donald Olding Hebb. *The organization of behavior: A neuropsychological theory*. Psychology press, 2005.
- [15] John J Hopfield. “Neural networks and physical systems with emergent collective computational abilities.” In: *Proceedings of the national academy of sciences* 79.8 (1982), pp. 2554–2558.
- [16] Jerry Yao-Chieh Hu, Dennis Wu, and Han Liu. “Provably optimal memory capacity for modern hopfield models: Transformer-compatible dense associative memories as spherical codes”. In: *arXiv preprint arXiv:2410.23126* (2024).
- [17] Jiayi Hu, Yongqi Pan, Jusen Du, Disen Lan, Xiaqiang Tang, Qingsong Wen, Yuxuan Liang, and Weigao Sun. “Comba: Improving Nonlinear RNNs with Closed-loop Control”. In: *arXiv preprint arXiv:2506.02475* (2025).
- [18] Weizhe Hua, Zihang Dai, Hanxiao Liu, and Quoc Le. “Transformer quality in linear time”. In: *International conference on machine learning*. PMLR. 2022, pp. 9099–9117.
- [19] Kazuki Irie, Imanol Schlag, Robert Csordas, and Jurgen Schmidhuber. “Going beyond linear transformers with recurrent fast weight programmers”. In: *Advances in neural information processing systems* 34 (2021), pp. 7703–7717.

- [20] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. *Muon: An optimizer for hidden layers in neural networks*. 2024. URL: <https://kellerjordan.github.io/posts/muon/>.
- [21] M. Karami and V. Mirrokni. *Lattice: Learning to Efficiently Compress the Memory*. 2025.
- [22] Mahdi Karami, Ali Behrouz, Praneeth Kacham, and Vahab Mirrokni. “TRELLIS: Learning to Compress Key-Value Memory in Attention Models”. In: *Second Conference on Language Modeling*. 2025. URL: <https://openreview.net/forum?id=r61s1FNYlj>.
- [23] Diederik P Kingma and Jimmy Ba. “Adam: A method for stochastic optimization”. In: *arXiv preprint arXiv:1412.6980* (2014).
- [24] Dmitry Krotov. “Hierarchical associative memory”. In: *arXiv preprint arXiv:2107.06446* (2021).
- [25] Dmitry Krotov and John J Hopfield. “Dense associative memory for pattern recognition”. In: *Advances in neural information processing systems* 29 (2016).
- [26] Xiaoyu Li, Yuanpeng Li, Yingyu Liang, Zhenmei Shi, and Zhao Song. “On the expressive power of modern hopfield networks”. In: *arXiv preprint arXiv:2412.05562* (2024).
- [27] Yi Heng Lim, Qi Zhu, Joshua Selfridge, and Muhammad Firmansyah Kasim. “Parallelizing non-linear sequential models over the sequence length”. In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=E34AlVLN0v>.
- [28] Bo Liu, Rui Wang, Lemeng Wu, Yihao Feng, Peter Stone, and Qiang Liu. “Longhorn: State space models are amortized online learners”. In: *arXiv preprint arXiv:2407.14207* (2024).
- [29] Ilya Loshchilov and Frank Hutter. “Decoupled weight decay regularization”. In: *arXiv preprint arXiv:1711.05101* (2017).
- [30] Carlo Lucibello and Marc Mézard. “Exponential capacity of dense associative memories”. In: *Physical Review Letters* 132.7 (2024), p. 077301.
- [31] William Merrill, Jackson Petty, and Ashish Sabharwal. “The Illusion of State in State-Space Models”. In: *Forty-first International Conference on Machine Learning*. 2024. URL: <https://openreview.net/forum?id=QZgo9JZpLq>.
- [32] Tsendsuren Munkhdalai, Alessandro Sordani, Tong Wang, and Adam Trischler. “Metalearned neural memory”. In: *Advances in Neural Information Processing Systems* 32 (2019).
- [33] Tsendsuren Munkhdalai and Hong Yu. “Neural semantic encoders”. In: *Proceedings of the conference. Association for Computational Linguistics. Meeting*. Vol. 1. NIH Public Access. 2017, p. 397.
- [34] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, et al. “Rwkv: Reinventing rnns for the transformer era”. In: *arXiv preprint arXiv:2305.13048* (2023).
- [35] Bo Peng, Daniel Goldstein, Quentin Anthony, Alon Albalak, Eric Alcaide, Stella Biderman, Eugene Cheah, Teddy Ferdinan, Haowen Hou, Przemysław Kazienko, et al. “Eagle and Finch: RWKV with Matrix-Valued States and Dynamic Recurrence”. In: *arXiv preprint arXiv:2404.05892* (2024).
- [36] Bo Peng, Ruichong Zhang, Daniel Goldstein, Eric Alcaide, Haowen Hou, Janna Lu, William Merrill, Guangyu Song, Kaifeng Tan, Saiteja Utpala, et al. “Rwkv-7” goose” with expressive dynamic state evolution”. In: *arXiv preprint arXiv:2503.14456* (2025).
- [37] Hao Peng, Jungo Kasai, Nikolaos Pappas, Dani Yogatama, Zhaofeng Wu, Lingpeng Kong, Roy Schwartz, and Noah A. Smith. “ABC: Attention with Bounded-memory Control”. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Smaranda Muresan, Preslav Nakov, and Aline Villavicencio. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 7469–7483. DOI: 10.18653/v1/2022.acl-long.515. URL: <https://aclanthology.org/2022.acl-long.515/>.
- [38] DL Prados and SC Kak. “Neural network capacity using delta rule”. In: *Electronics Letters* 25.3 (1989), pp. 197–199.
- [39] Zhen Qin, Weigao Sun, Dong Li, Xuyang Shen, Weixuan Sun, and Yiran Zhong. “Various Lengths, Constant Speed: Efficient Language Modeling with Lightning Attention”. In: *Forty-first International Conference on Machine Learning*. 2024.
- [40] Zihan Qiu, Zekun Wang, Bo Zheng, Zeyu Huang, Kaiyue Wen, Songlin Yang, Rui Men, Le Yu, Fei Huang, Suozhi Huang, et al. “Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free”. In: *arXiv preprint arXiv:2505.06708* (2025).
- [41] Hubert Ramsauer, Bernhard Schäfl, Johannes Lehner, Philipp Seidl, Michael Widrich, Lukas Gruber, Markus Holzleitner, Thomas Adler, David Kreil, Michael K Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. “Hopfield Networks is All You Need”. In: *International Conference on Learning Representations*. 2021. URL: <https://openreview.net/forum?id=tL89RnzIiCd>.

- [42] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. “Linear transformers are secretly fast weight programmers”. In: *International Conference on Machine Learning*. PMLR. 2021, pp. 9355–9366.
- [43] Jürgen Schmidhuber. “Learning to control fast-weight memories: An alternative to dynamic recurrent networks”. In: *Neural Computation* 4.1 (1992), pp. 131–139.
- [44] Jürgen Schmidhuber. “Reducing the ratio between learning complexity and number of time varying variables in fully recurrent nets”. In: *ICANN’93: Proceedings of the International Conference on Artificial Neural Networks Amsterdam, The Netherlands 13–16 September 1993* 3. Springer. 1993, pp. 460–463.
- [45] Mark Schöne, Babak Rahmani, Heiner Kremer, Fabian Falck, Hitesh Ballani, and Jannes Gladrow. “Implicit Language Models are RNNs: Balancing Parallelization and Expressivity”. In: *arXiv preprint arXiv:2502.07827* (2025).
- [46] Julien Siems, Timur Carstensen, Arber Zela, Frank Hutter, Massimiliano Pontil, and Riccardo Grazi. “DeltaProduct: Increasing the Expressivity of DeltaNet Through Products of Householders”. In: *arXiv preprint arXiv:2502.10297* (2025).
- [47] Jimmy T.H. Smith, Andrew Warrington, and Scott Linderman. “Simplified State Space Layers for Sequence Modeling”. In: *The Eleventh International Conference on Learning Representations*. 2023. URL: <https://openreview.net/forum?id=Ai8Hw3AXqks>.
- [48] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, et al. “Learning to (learn at test time): Rnns with expressive hidden states”. In: *arXiv preprint arXiv:2407.04620* (2024).
- [49] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. “Retentive network: A successor to transformer for large language models”. In: *arXiv preprint arXiv:2307.08621* (2023).
- [50] Matteo Tiezzi, Michele Casoni, Alessandro Betti, Tommaso Guidi, Marco Gori, and Stefano Melacci. “On the resurgence of recurrent models for long sequences: Survey and research opportunities in the transformer era”. In: *arXiv preprint arXiv:2402.08132* (2024).
- [51] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).
- [52] Ke Alexander Wang, Jiaxin Shi, and Emily B. Fox. *Test-time regression: a unifying framework for designing sequence models with associative memory*. 2025. arXiv: 2501.12352 [cs.LG]. URL: <https://arxiv.org/abs/2501.12352>.
- [53] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. “Gated Linear Attention Transformers with Hardware-Efficient Training”. In: *International Conference on Machine Learning*. PMLR. 2024, pp. 56501–56523.
- [54] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. “Parallelizing Linear Transformers with the Delta Rule over Sequence Length”. In: *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. 2024.
- [55] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. “Parallelizing linear transformers with the delta rule over sequence length”. In: *Advances in neural information processing systems* 37 (2024), pp. 115491–115522.
- [56] Tianyuan Zhang, Sai Bi, Yicong Hong, Kai Zhang, Fujun Luan, Songlin Yang, Kalyan Sunkavalli, William T Freeman, and Hao Tan. “Test-time training done right”. In: *arXiv preprint arXiv:2505.23884* (2025).
- [57] Yu Zhang, Songlin Yang, Rui-Jie Zhu, Yue Zhang, Leyang Cui, Yiqiao Wang, Bolun Wang, Freda Shi, Bailin Wang, Wei Bi, et al. “Gated slot attention for efficient linear-time sequence modeling”. In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 116870–116898.

A 相关工作

现代线性循环神经网络 由于变换器（Transformer）的二次复杂度，近期发展中的替代架构引起了关注，这推动了高效循环替代方案的发展（Tiezzi 等, 2024）。该领域的初步进展始于 RetNet（Sun、Dong 等, 2023）、RWKV（Peng、Alcaide 等, 2023）和 S5（Smith 等, 2023）等模型，这些模型采用了与数据无关的转移矩阵，并结合了类似赫布（Hebbian）的更新机制。随后，第二代模型出现，在这些线性架构中引入了依赖输入的参数（例如，线性循环神经网络（RNN）（Hasani 等, 2023; Smith 等, 2023）、RWKV6（Peng、Goldstein 等, 2024））。这些模型还探索了更具表达力的记忆更新规则，尤其是基于增量规则（delta rule）的规则（Peng、Zhang 等, 2025; Schlag 等, 2021; Yang、Wang、Zhang 等, 2024b; Yang、Wang、Shen 等, 2024; Liu 等, 2024）。这一研究方向的进一步演进将这些记忆架构扩展到更深的模型，同时利用类似增量规则的更新机制（Sun、Li 等, 2024）或基于数据依赖动量的更新规则，并带有遗忘门控（Behrouz、Zhong 等, 2025）。最近，为了提升基于增量规则的序列模型的性能，Siems 等（2025）提出对每个标记应用多次梯度下降更新，从而产生更具表达力的序列模型，特别是在状态跟踪任务中。除了上述快速线性循环序列模型外，一些研究还关注了具有非线性循环的循环神经网络（Behrouz、Zhong 等, 2025; Behrouz、Razaviyayn 等, 2025b; Behrouz、Li 等, 2025; Csordás 等, 2024; Merrill 等, 2024; Lim 等, 2024; Schöne 等, 2025; Karami 和 Mirrokni, 2025; Gonzalez 等, 2024），以及如何加快其训练速度（Gonzalez 等, 2024; Lim 等, 2024; Schöne 等, 2025）。

快速权重程序 将线性层概念化为键值联想记忆系统可追溯至霍普菲尔德网络（Hopfield Networks）（Hopfield 1982）。这一概念随后在快速权重编程器的背景下得到发展，其中动态快速程序被集成到循环神经网络中，作为可写的记忆存储（Schlag et al. 2021; Schmidhuber 1992; Schmidhuber 1993）。在此类系统的学习范式中，赫布学习（Hebbian Learning）（Hebb 2005）和德尔塔规则（Delta Rule）（Prados et al. 1989）最为突出。这两种学习规则在现有文献中均得到了广泛研究（Munkhdalai and Yu 2017; Schmidhuber 1992; Munkhdalai, Sordoni, et al. 2019; Schlag et al. 2021; Irie et al. 2021; Yang, Wang, Zhang, et al. 2024b; Yang, Wang, Shen, et al. 2024）。

霍普菲尔德网络 本文的公式在架构上基于联想记忆的广义概念，其主要目标是学习键与值之间的底层映射。霍普菲尔德（Hopfield）（1982）关于霍普菲尔德网络的开创性工作引入了最早明确基于联想记忆的神经架构之一，通过最小化能量函数来存储键值对。尽管传统霍普菲尔德网络近年来适用性有所下降，主要由于向量值记忆容量的约束及其能量函数的性质，但若干当代研究致力于通过各种方法增强其容量。这些包括 Krotov（2021）、Li 等人（2024）以及 Krotov 和 Hopfield（2016）的工作。值得注意的是，对这些模型能量函数的扩展，通常引入指数核函数，已被探索（Krotov and Hopfield 2016; Lucibello et al. 2024）。此外，这些现代化霍普菲尔德网络与变换器（Transformer）架构之间的关系已成为近期研究的主题（Ramsauer et al. 2021; Hu, Wu, et al. 2024）。

B QK 投影与记忆有界变换器的联系

回顾本文的 QK 投影检索过程，首先将查询向量投影到存储键的空间中，然后通过对记忆模块的前向传递检索其对应的值。具体而言，给定查询 q_t 和存储键 $\{k_i\}_{i=1}^t$ ，输出对应于查询 q_t 可计算为：

$$o_t = f \left(W_t, \sum_{\tau=1}^t \frac{k_\tau k_\tau^\top}{\|k_\tau\|^2} q_t \right). \quad (8)$$

给定键的归一化，即 $\|k_\tau\|_2 = 1$ ，该公式可重写为：

$$o_t = f \left(W_t, \sum_{\tau=1}^t k_\tau k_\tau^\top q_t \right), \quad (9)$$

其中第二个元素， $\sum_{\tau=1}^t k_\tau k_\tau^\top a_t$ ，等价于对查询 a_t 在线性记忆模块上的一次简单前向传递，其递推关系 of $\mathcal{M}'_t = \sum_{\tau=1}^t k_\tau k_\tau^\top$ 为：

$$\mathcal{M}'_t = \mathcal{M}'_{t-1} + k_t k_t^\top. \quad (10)$$

这种 QK 投影的表述方式使我们联想到记忆受限变换器（Memory Bounded Transformers）的两阶段处理过程（Peng, Kasai, et al. 2022; Zhang, Yang, et al. 2024; Karami, Behrouz, et al. 2025），其中在简单的线性注意力形式下（Peng, Kasai, et al. 2022），检索过程可以写为：

$$W_t = W_{t-1} + \varphi_t v_t^\top, \quad (11)$$

$$o_t = W_t \text{softmax} \left(\left(\sum_{\tau=1}^t k_\tau \varphi_\tau^\top \right) q_t \right). \quad (12)$$

与上述 ABC 的两阶段处理过程（Peng, Kasai, et al. 2022）相比，我们的 QK 投影方法既适用于深度记忆也适用于线性记忆。此外， φ_t 以及 k_t 的参数是共享的，因此学习过程显著简化，使模型对新数据/任务的适应性更强。而且，当在局部记忆中使用该投影时，我们仅从记忆的“重置”状态开始求和，而不是从 $\tau = 1$ 开始。

C QK 投影的高效实现

本节详细介绍了 QK 投影机制的高效、可并行化实现。我们证明该投影可以集成到分块训练范式中，而不会引入顺序瓶颈，从而保持 TNT 架构的训练效率。

QK 投影依赖于投影矩阵 $\mathcal{M}_t$ ，该矩阵在当前局部记忆分片（长度为 S_L ）内累积键的外积。假设键已归一化（ $\|k_\tau\| = 1$ ），该矩阵由以下递推定义：

$$\mathcal{M}_t = \begin{cases} k_t k_t^\top & \text{if } t \equiv 1 \pmod{S_L} \\ \mathcal{M}_{t-1} + k_t k_t^\top & \text{otherwise} \end{cases}$$

该规则确保投影状态 $\mathcal{M}_t$ 在每个分片开始时重置，与局部记忆状态 W_t 的重置一致。然后局部记忆检索计算为 $f(W_t, \mathcal{M}_t a_t)$ 。

分块并行计算。为保持训练效率， $\mathcal{M}_t$ 以分块并行方式计算。对于大小为 C_L 的块内任意时间步 t ，投影矩阵可分解为两个分量：

$$\mathcal{M}_t = \underbrace{\mathcal{M}_{\xi(t, C_L)-1}}_{\text{Carry-over State}} + \underbrace{\sum_{\tau=\xi(t, C_L)}^t k_\tau k_\tau^\top}_{\text{块内求和}}$$

第一项是来自前一块的最终状态，被传递过来。第二项，即块内求和，通过对序列外积 $\{k_\tau k_\tau^\top\}$ 执行并行前缀和（扫描）操作，同时高效计算块内所有步骤。

该实现保持了端到端并行性。块间传递的状态是单个恒定大小的矩阵 ($d \times d$)，开销极小。周期性重置通过在分片边界重新初始化该传递状态来处理。因此，QK 投影在增强模型检索机制的同时，并未损害 TNT 架构所依赖的训练效率。

D TNT 适用性

本文重点展示了 TNT 的有效性和效率，为清晰起见，采用了一个简单的记忆模块，其内循环通过梯度下降进行优化。然而，TNT 方案适用于

不同的深度记忆和非线性架构。例如，可以将 Titans (Behrouz, Zhong 等, 2025) 或 Mamba2 (Dao 和 Gu, 2024) 中的门控公式适配到每个局部记忆以及全局记忆。另一个潜在的探索方向是在内循环目标中引入闭环反馈，如 Hu, Pan 等 (2025) 所做。类似地，可以采用更具表达力的优化器作为内循环优化器，例如带动量的梯度下降、AdamW (Kingma 等, 2014) 或 muon (Jordan 等, 2024)，正如 Behrouz, Li 等 (2025) 和 Zhang, Bi 等 (2025) 所做。虽然探索所有这些与 TNT 的组合是一个有前景的方向，但为清晰和篇幅所限，我们将其留待未来研究。

E TNT 泛化公式

TNT 架构可以推广到一个包含一个全局记忆和 N 个并行局部记忆的分层系统。这使得模型能够同时捕获多个时间尺度和分辨率的信息。每个局部记忆，记为 $W^{(i)}$ ，其中 $i \in \{1, \dots, N\}$ ，以其各自不同的块大小 C_{L_i} 、分片长度 S_{L_i} 和可学习的初始状态 $W_{\text{init}}^{(i)}$ 运行。

E.1 广义记忆更新

更新规则扩展如下：单个全局记忆顺序演化，而 N 个局部记忆并行更新，每个都有其独立的调度。

全局记忆更新。全局记忆状态 V 以较大的块大小 C_G 顺序更新，与基本情况相同。

$$V_{(k+1)C_G} \leftarrow V_{kC_G} - \sum_{t=kC_G}^{(k+1)C_G} \eta_t \nabla_V \mathcal{L}(f(V_{kC_G}, k_t), v_t) \quad (13)$$

N 个局部记忆更新。每个 N 个局部记忆 $W^{(i)}$ 并行运行。每个记忆的状态根据其特定的分片长度 S_{L_i} 周期性重置，从而实现多分辨率上下文并行。

$$W_t^{(i)} \leftarrow \begin{cases} W_{\text{init}}^{(i)} & \text{if } 0 \equiv t \pmod{S_{L_i}} \\ W_{t-1}^{(i)} - \sum_{\tau=\xi(t, C_{L_i})}^t \eta_\tau \nabla_{W^{(i)}} \mathcal{L}(f(W_{\xi(t, C_{L_i})}^{(i)}, k_\tau), v_\tau) & \text{Otherwise} \end{cases} \quad (14)$$

其中 $i = 1, \dots, N$ 。

E.2 广义记忆检索

在检索过程中，最终输出是全局记忆与所有 N 局部记忆输出的组合。全局记忆使用原始查询 q_t ，而每个局部记忆使用针对其特定上下文窗口定制的 Q-K 投影，该窗口由其块大小 C_{L_i} 决定。

TNT 广义检索规则。分层记忆通过求和每个记忆模块的贡献来进行检索。

$$o_t = f(V_{\xi(t, C_G)}, q_t) + \sum_{i=1}^N f\left(W_t^{(i)}, \sum_{\tau=\xi(t, C_{L_i})}^t \frac{k_\tau k_\tau^\top}{\|k_\tau\|^2} q_t\right) \quad (15)$$

该公式允许网络将来自全局记忆的长程依赖与来自多样化局部记忆集合的细粒度、并行处理信息相结合，每个局部记忆专注于不同的时间模式。

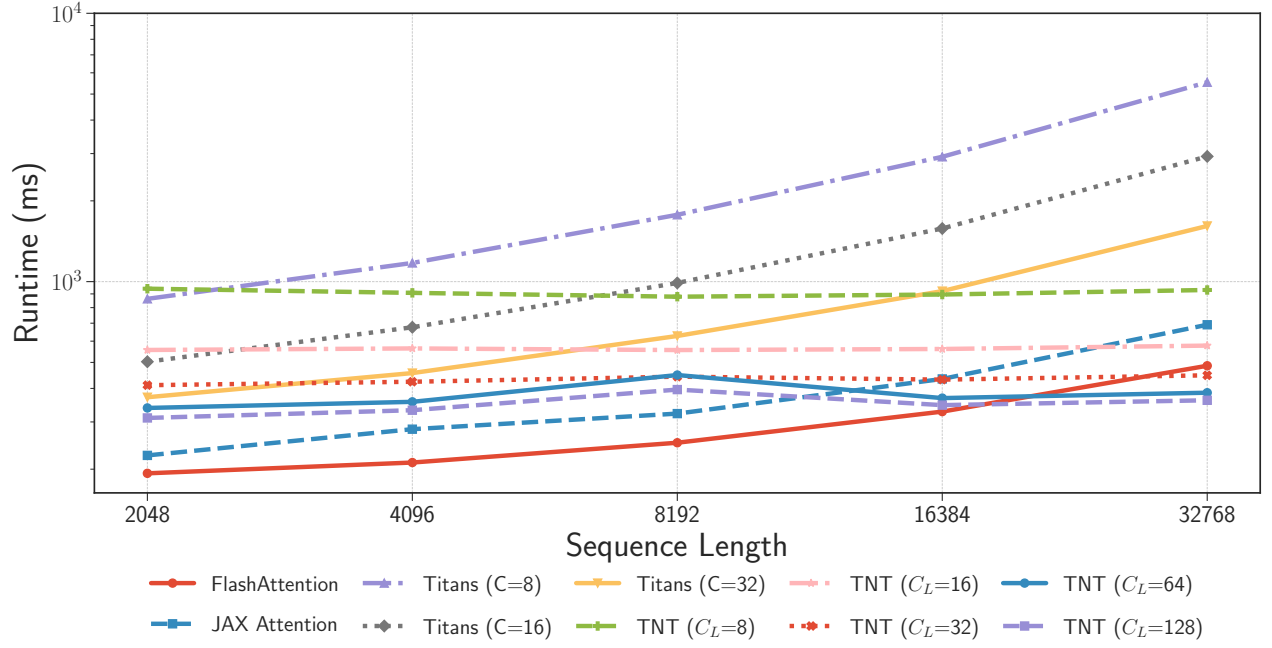


图 5: 不同模型和实现在不同序列长度下的运行时间比较，每批固定 0.5M 个标记。

表 4: 在 10B 标记上训练的 150M 参数模型的训练时间。对于 TNT 模型，全局块大小固定为 $C_G = 2048$ ， $\tilde{C}_L$ 表示局部记忆模块的块大小集合。

Model	C or C_L	Training Time (hrs)
150M params / 10B tokens		
Transformer (w/o gating)	-	0.80
Transformer (w gating)	-	0.82
TTT (2024)	256	1.69
Titans (2025)	256	1.99
Titans	8	8.44
TNT Stage 1		
TNT Stage 1	{8}	3.06
	{8,16}	4.24
	{4,8,16}	5.00
	{4,8,16,32}	5.55
TNT Stage 2		
TNT Stage 2	{1}	0.15
	{2,4}	0.23
	{2,4,8}	0.26
	{2,4,8,16}	0.46